

SLO-05: SLOs as Code

Series: SLO — Service Level Objectives | **Notebook:** 5 of 5 | **Created:** June 2026
| **Last Updated:** 06/16/2026

Overview

A validated SLO that lives only in the UI is one accidental click from gone, and impossible to reproduce across environments. This notebook covers promoting SLOs into version control: the `builtin:slo` schema, the `dynatrace_slo_v2` Terraform resource, the Monaco alternative, and the API path — with a deliberate emphasis on verifying field names at the source rather than copying a payload that may be stale.

Table of Contents

- 1. [Why Version SLOs](#)
- 2. [The Schema and Terraform Resource](#)
- 3. [Terraform Worked Example](#)
- 4. [Monaco Alternative](#)
- 5. [API and CI/CD](#)

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3 with the SLO app

Requirement	Details
Tooling	Terraform with the <code>dynatrace-oss/dynatrace</code> provider, or Monaco
Auth	Platform Token or OAuth client per the AUTOM-04 auth-scheme guidance
Prior reading	SLO-02 (the SLI query you will codify), AUTOM-04 / AUTOM-07 (provider auth, CI/CD)

1. Why Version SLOs

SLOs are configuration, and configuration that matters belongs in source control:

- **Reproducible across environments** — the same SLO in dev, staging, and prod from one definition.
- **Reviewable** — a target change goes through a PR, not a quiet UI edit.
- **Recoverable** — an accidental deletion is a `terraform apply` away from restored.

The workflow mirrors AIOPS-02's stance on detectors: **prototype in the app, promote to code once it matters.**



2. The Schema and Terraform Resource

Surface	Identifier
Settings 2.0 schema	<code>builtin:slo</code>
Terraform resource	<code>dynatrace_slo_v2</code>
Monaco	<code>builtin:slo</code> schema via <code>--settings-schema</code>

The `dynatrace_slo_v2` resource captures the same four decisions from SLO-01: an SLI query, a target, an optional warning level, and an evaluation window.

Verify the exact argument names against the provider docs before you write HCL. The provider schema evolves between releases, and copying a stale payload is precisely how the field-authored alerting document ended up with an SLO API body that would not apply. The worked example below is illustrative — treat the [resource docs](#) as authoritative for current field names.

```
# Illustrative dynatrace_slo_v2 – confirm argument names at the provider
registry
# (schema evolves between provider versions; this is the shape, not a
guaranteed-current payload)
resource "dynatrace_slo_v2" "web_availability_30d" {
  name          = "Web Service Availability – 30d"
  description    = "Request success ratio for the critical web service, rolling
30 days"
  enabled       = true

  # DQL-based SLI: good / total as a percentage
  metric_expression = "(100) * (builtin:service.successCount) /
(builtin:service.requestCount)"

  evaluation_type = "AGGREGATE"
  filter          = "type(\"SERVICE\")"

  target  = 99.5
  warning = 99.9

  # rolling 30-day window
  timeframe = "-30d"
}
```

3. Terraform Worked Example — Notes

A few things the example encodes:

- **target** and **warning** are the goal and the early-warning line from SLO-01 §4. Warning above target surfaces "getting close" before an actual breach.
- **The SLI expression** is the codified form of the good/total query you validated in SLO-02. Some provider versions take a metric selector, others a DQL query field — this is the argument most likely to differ by version, so confirm it.
- **filter / scope** binds the SLO to a service (or tag, or management zone). An unscoped SLO measures the whole environment, which is rarely what you want.
- **State and auth** follow the same rules as every other Dynatrace Terraform resource — see AUTOM-04 for Platform-Token vs classic-token routing and AUTOM-09 for state-backend setup.

4. Monaco Alternative

If your shop standardises on Monaco rather than Terraform, the same SLO is a `builtin:slo` settings object:

```
configs:
  - id: web-availability-30d
    type:
      settings:
        schema: builtin:slo
        scope: environment
      config:
        template: web-availability-30d.json
        skip: false
```

The JSON template carries the SLI, target, warning, and window. As with Terraform, validate the SLI query in a notebook first, then export the working SLO from the app with `monaco download` to capture the exact current JSON shape rather than hand-writing it.

5. API and CI/CD

For direct API automation, use the **current SLO endpoint**, not the deprecated v1 SLO API. Rather than reproduce a request body that may drift, generate it from a working SLO:

1. Create and validate one SLO in the app.
2. Read it back via the API (or `monaco download` / `terraform-provider-dynatrace -export`) to capture the exact current schema.
3. Template that shape for the rest.

This "export a known-good object" approach is the antidote to stale-payload errors. For the full CI/CD pattern — plan on PR, gated apply, three-way validation — see AUTOM-07 (pipelines) and AUTOM-96 (GitHub Actions LAB); the SLO resource slots into exactly the same pipeline as any other Dynatrace config.

Sources: [Service-Level Objectives \(DT docs\)](#), [dynatrace_slo_v2 resource \(Dynatrace provider docs\)](#), [Monaco configuration \(DT docs\)](#). **Softened:** exact `dynatrace_slo_v2` argument names are version-dependent — verify at the registry; the HCL here is illustrative shape, not a validated payload.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.